Technical Design Blueprint: Cloud-Native Telemetry & WFM Data Platform

Olivera Sazos, Apr 2026

TECHNICAL DESIGN & ENGINEERING OVERVIEW DRAFT

This technical document details a cloud-native data platform designed to consolidate contact centre telemetry into a unified analytics layer. By integrating telephony metrics, workforce management (WFM) schedules, and organisational hierarchies, the solution synchronises performance data with operational structures to enable multi-dimensional trend analysis.

Design Approach

The platform utilises a Medallion Architecture to manage the data lifecycle. Raw ingestion ensures full auditability and replicability, while subsequent transformation layers enforce schema validation and data quality. The final refined layer provides high-performance, datasets optimised for downstream BI consumption and strategic decision-making.

Landing Temporary Layer (only for the new files to assure only they are processed)

Permanent Layer

- Injection Layer: Raw files, timestamp so not to be overwritten if the same name
- Bronze Layer: Structured tables
- Silver Layer: Transformed
- Gold Layer: Business-ready reporting datasets

Data Sources

- API: Agent telephony performance (daily grain)
- CSV Files: Agent schedules (daily ingestion)
- Excel Files: Team hierarchy (periodic updates)

Processing Flow

- Data is extracted using Python-based ingestion processes
- Raw data is stored in the Ingestion layer (original format) and in Bronze layer (after injection)
- Data is cleaned and standardised in the Silver layer in Landing Area
- Business logic is applied in the Gold layer
- Analytics consume Gold-layer datasets

1. API INGESTION FRAMEWORK DESIGN

The agent performance API delivers daily summaries at the grain of (date, agent_id). The design must handle authentication token expiry, pagination, historical corrections, and idempotent reprocessing.

START
Load configuration (API URL, auth URL, client credentials, start_date, end_date, last_updated, token lifetime)
Request initial token → authenticate via auth API

Receive access token + token timestamp
  Authentication handling (initial token generation + session setup)

START INGESTION LOOP

Check token expiry **(current time - token time > 60 minutes?)**

IF expired, refresh token **(Authentication handling, token lifecycle + refresh logic)**

Build rAPI equest **(Include auth token, Include date range and watermark, Include cursor)**
Send API request **(Incremental extraction strategy - last_updated controls incremental loads, Pagination handling -cursor included for multi-page retrieval)**

Send API request (GET)

Check response status
if Error
  401 (Auth expired) - refresh token and retry
  429 (Rate limit 1000) - wait / retry later
  5xx (Server error) - retry request (transient server failure recovery)
  timeout → retry request, (slow/unresponsive API)
  other error → stop pipeline (non-recoverable failure handling)

if success (200 OK)
  Extract data, **Monitoring & validation controls (can track record counts, completeness per batch)**
  Collect records into all_records

Read next_cursor, **Pagination handling (controls multi-page traversal via API cursor)**
if cursor exists → continue loop (next page)
if no cursor EXIT LOOP

Return records
Monitoring & validation controls (final reno cursso cord count check, ingestion completeness verification)

END

Error handling and retry strategy

- Ensures the pipeline is resilient when calling external APIs by handling failures in a controlled way.
- It distinguishes between permanent errors (which stop the pipeline) and temporary issues (which trigger retries).

- The strategy improves reliability by retrying transient failures, handling token expiry, respecting rate limits, and preventing the system from hanging or failing unnecessarily.

Reprocessing

- The script calls the same API every time it runs
- Reprocessing is enabled by the request parameters, especially the date range (start_date / end_date) and last_updated

2. SCHEDULES ENGINEERING

CSV to be loaded daily. The load process to depend on storage architecture of files.
Overlaps to be handled in Silver layer.
This is the simplest algorithm which could be enriched with slightly different logic, using different requirements (e.g. priorities)

The Overlap Resolution Algorithm Draft – Basic

Group data by employee and date so each person's daily records are processed separately.
Within each group, sort records by start time so they are in chronological order.
Go through each record one by one.
For the first record, simply add it to the cleaned list.
For each next record, compare it with the previous one that was kept:

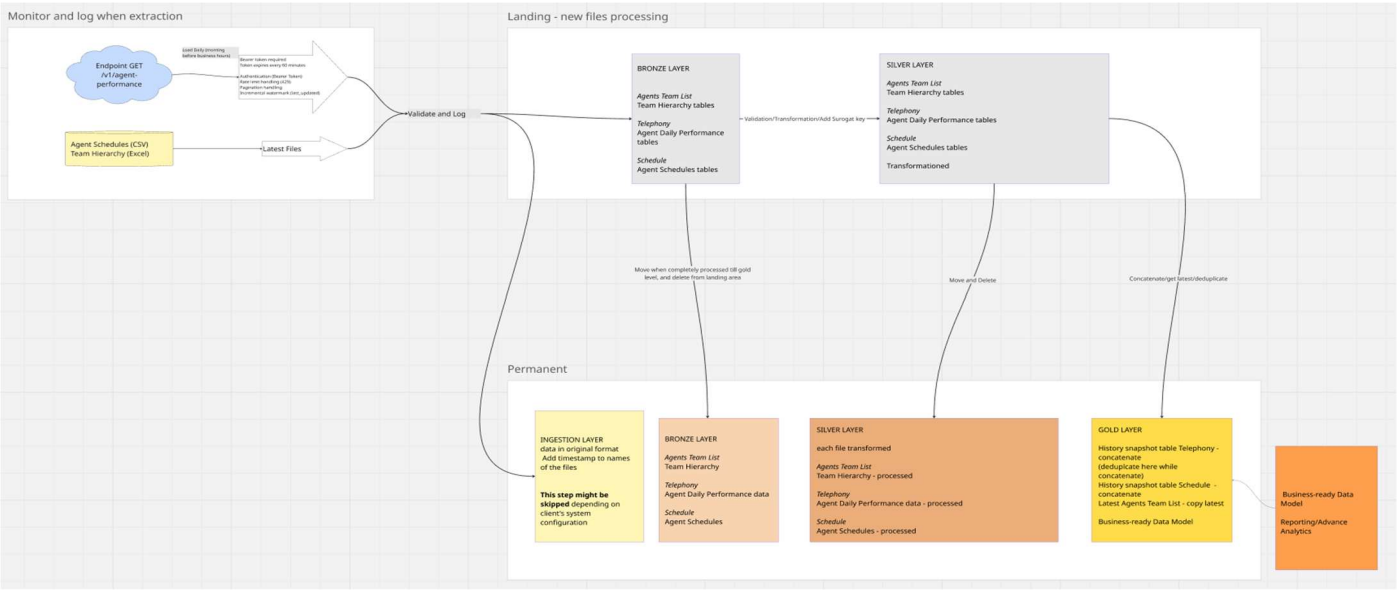
- If the previous record ends after the current record starts, there is an overlap.
- In that case, shorten the previous record so it ends at the current record's start time.

Then add the current record to the cleaned list.
Repeat this process until all records in the group are processed.

3. ARCHITECTURAL DESIGN

The architecture follows a medallion lakehouse pattern (Bronze → Silver → Gold)

3.1 High Level Architecture Draft



3.2 Key Design Decisions

Data Architecture (Medallion Framework)

- Use landing/staging area for new files and their processing.
- Move files from Landing to Permanent storage only after the full pipeline reaches Gold level.
- Immutable Storage, maintain an unchangeable Bronze layer to ensure logic can be replayed at any time.
- Hierarchy-first processing, process Team Hierarchy first to create Surrogate Keys required by other datasets.

3.3 Validation & Logging

- Processing checks, verify dates, IDs, and name clarity in Silver layer.
- Halt or alert if invalid records (nulls/duplicates) exceed a 5–10% threshold.
- Identity resolution, match Telephony data (no employee_id present in data) to IDs via standardised (uppercase) name lookups.
- Historical tracking: use Surrogate Keys (ID + Date) to link calls to the correct manager at that point in time

3.4 Monitoring

- Execution Logs: The system tracks processing time and error flags for every run.

3.5 Gold Layer & Reporting, Historical Snapshots

- Telephony and Schedule data are concatenated into History Snapshot Tables in the Gold Layer to support trend analysis.
- Business-Ready model, dimensions and fact tables to connect to dim tables are conformed via Surrogate Keys and date to provide Star Schema for Reporting and Advanced Analytics.

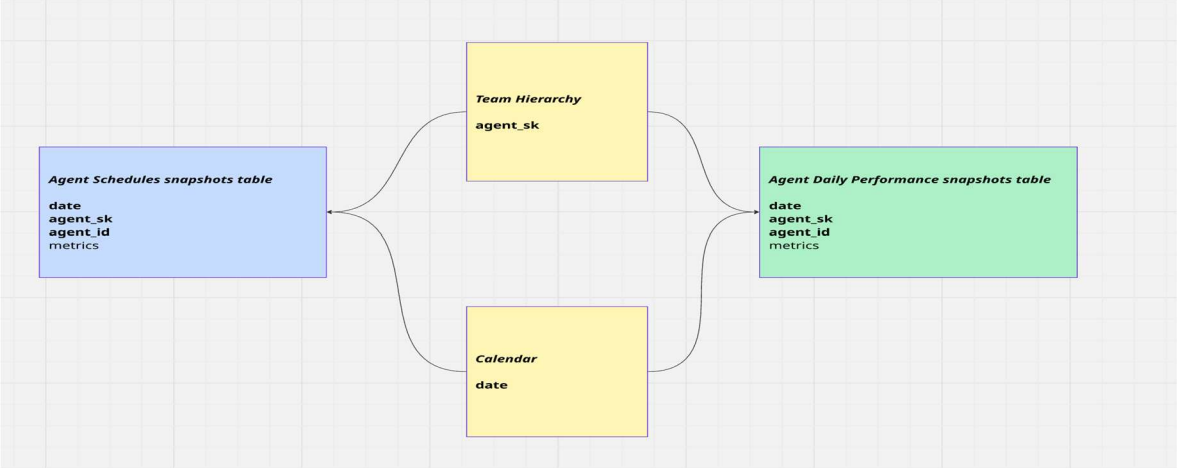
3.6 Risk Register

Risk ID	Risk Description	Impact	Mitigation Strategy
R01	API Rate Limiting, ingestion fails or stalls due to HTTP 429 errors during large historical re-loads.	Low	Add delays between retries that increase each time.
R02	Schema Evolution, new fields added to the API or Excel files break existing transformation pipelines.	Medium	To be resolved in Silver layer
R03	Manual File Errors: CSV or Excel files contain corrupt formatting, missing headers, or invalid date types.	High	To be resolved in Bronze or Silver layer. Reject file or quarantine, create error flags in Silver Layer
R04	Incomplete API Payload, API returns partial responses, leading to missing agent records for specific dates.	Medium	Validate record counts against expected thresholds. Ideally to be solved with flags in Silver layer

4. DIMENSIONAL MODEL DIAGRAM

The dimensional model uses a star schema to support reporting by date, agent, team, manager, and financial year. Slowly Changing Dimensions are implemented for the agent-team hierarchy, since agents change managers and regions over time.

The star schema centres on two fact tables; each connected to shared dimension tables



Key Design Decisions & Justifications

Design Choices

- Star Schema, fact tables (Schedules/Performance) connect to shared dimensions (Hierarchy/Calendar) for reporting.
- The Team Hierarchy tracks manager changes over time using unique Surrogate Keys.
- Daily Snapshots, performance and schedules are recorded daily to preserve "as-was" historical data.
- Surrogate Key Grain: all tables link via agent_sk to keep history stable.
- History is changed only (to be clarified) if duplicate records come in Telephony table (agent id and date grain).

Why This Works

- Historical Accuracy, you can see who an agent's manager was on a specific day in the past, not just who it is today.
- Performance: BI tools process this structure much faster.
- Planned vs. Actual, the shared Calendar and SK allow to compare an agent's scheduled hours against actual performance.

Reporting

- Trend Analysis: Daily snapshots make it simple to track performance growth over time.
- Consistent Reporting: Centralized dimensions ensure everyone sees the same team structures and date totals.